ADB over VPN — Full Device Bridge Design (Access · Debug · Connect · Flash)

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Design reference — the in-depth, deep-research-backed phased plan behind PLAN.md §5 Phase 7 and the committed autonomous test ../../../tests/vpn_lan/adb_over_vpn.sh. DESIGN + RESEARCH ONLY — this document ships no code, touches no data-plane, deploys nothing. **Authority:** Inherits constitution/Constitution.md per §11.4.35. Deep-research per §11.4.8 / §11.4.99 / §11.4.150 (every external claim cited by URL + access date, or marked **INFERENCE** per §11.4.6). Target-hardware-safety per §11.4.133; operator-gating per §11.4.122 / §11.4.66; single-serial safety per §11.4.174. **Companion:** plan PLAN.md (§2 routing map, §3 bridge contract, §5 Phase 7) · topology architecture.md (§1 diagram, §2 primitive table) · containerization containerization.md + vpn_lan_containers.yaml (the vpn-lan-adb-server service) · the proving test ../../../tests/vpn_lan/adb_over_vpn.sh (Phase 7). **Feature workstream:** feature/vpn-aware-dynamic-routing (§11.4.167).

---

0. How to read this document

This is the design that the committed test tests/vpn_lan/adb_over_vpn.sh implements against. The test is authoritative for what runs **now** (bridge-down honest-SKIP + the live checks it drives); this document is the deeper "why + how + what next" behind it. Where the two touch the same fact (the routed-5555 model, the central adb-server model, the usbip flash boundary, the single-serial §11.4.174 cleanup rule), this document **restates and never contradicts** the committed test.

Every external claim is either **cited** (URL + access date 2026-07-01, listed in the Sources verified footer) or explicitly marked **INFERENCE** (§11.4.6) — an engineering deduction from the cited facts, not itself a cited fact. Nothing here is stated as proven-live: the live device paths are operator-gated (§8) and prove out only when the operator supplies the svord bridge connection.

---

1. Overview + the routing FACT (what is network-routable vs USB-bound)

The VPN is **L3-routed** (WireGuard + L2TP/PPP over `ppp0`; reachable subnet `10.0.0.0/8`; svord host `10.6.100.221` — recon FACT, `PLAN.md` §2). A Layer-3 VPN carries **unicast IP packets** between reachable subnets — nothing more. That single property sorts every ADB capability into exactly one of two buckets:

ADB capability	Wire protocol	Routable over the L3 VPN?	Primitive
Access / connect (`adb connect`)	TCP 5555 (device `adbd`)	**YES** — unicast TCP	**L3-ROUTE** over `ppp0`
Debug (`adb shell` / `logcat` / `pull` / `push` / `getprop`)	TCP 5555 (over the same connection)	**YES** — unicast TCP	**L3-ROUTE**
adb server↔client control	TCP 5037 (host-local)	N/A (stays on the adb-server host)	co-located with the route owner
Flash (`fastboot`)	**USB transport** (bootloader USB gadget)	**NO** — not an IP service on consumer devices	**usbip** (USB-over-IP)

Rule of thumb (FACT): *`adb` speaks TCP and routes; `fastboot` speaks USB on consumer bootloaders and must be carried by usbip; the adb server's own control port (5037) stays local to whichever host owns the route to the device.*

The fastboot **protocol** is defined to run "over USB or ethernet" (AOSP fastboot README) — but the ethernet transport requires a device/bootloader that exposes fastboot over TCP/UDP, which consumer Android bootloaders overwhelmingly do **not** during the bootloader stage. Hence remote flashing carries the **USB** transport over the network via usbip (Phase C). This is the honest, precise form of the committed test's "network fastboot is USB-bound" boundary (**INFERENCE** from the protocol spec + consumer-bootloader reality; the fastboot-over-ethernet capability is a cited FACT, its general non-exposure on consumer bootloaders is the INFERENCE).

---

2. adb client / server / device model (the FACT the phases build on)

adb is a three-part system (Android Debug Bridge (adb)):

1. **client** — the `adb …` command you invoke; sends commands.
2. **server** — a background process on the host that manages the client↔device conversation; **binds host TCP port 5037** and every client talks to it there. One server fronts **many** devices.
3. **daemon (`adbd`)** — runs on each device; the server talks to it. Over the network, `adbd` listens on **TCP 5555** once `adb tcpip 5555` has been run (Android ≤10) or wireless-debugging is enabled (Android 11+).

`adb devices` lists every managed device with its state; the states relevant here are `device` (usable), `offline` (connected but not responding), and `unauthorized` (RSA key not yet accepted) (adb docs). A network device appears with its `IP:PORT` as the serial, e.g. `10.6.100.55:5555  device`. Target disambiguation is `adb -s <serial> …` (overrides `$ANDROID_SERIAL`) (adb docs).

This model is why the **central adb-server** design works: one adb server on the host that owns the route to `10.0.0.0/8` can `adb connect` to every remote device's 5555, and helix_proxy-side clients disambiguate by serial (§ Phase B).

Phase A — ADB ACCESS + DEBUG over routed 5555 `[autonomous-provable]`

Goal: prove a device exposed on TCP 5555 on the remote `10/8` subnet is reachable + usable through the L3-routed gateway, with real captured evidence — `adb connect`, `adb shell getprop`, `adb logcat`, `adb pull/push`. This is the capability the committed test `adb_over_vpn.sh` drives (T7.1 connect, T7.3 debug).

Deep-research FACTs (cited):

- Classic path: `adb tcpip 5555` (run once over USB) puts `adbd` into TCP mode on port 5555, then `adb connect DEVICE_IP:5555` connects over the network; `adb devices` shows `DEVICE_IP:5555 device` (adb docs). **A device reboot disables `adb tcpip` mode** — it must be re-enabled (scrcpy connection doc; ProAndroidDev / scrcpy-adb-wifi).
- Once connected, `adb -s <serial> shell getprop ro.product.model` returns the device model string; `adb -s <serial> logcat`, `adb -s <serial> pull <remote> <local>`, `adb -s <serial> push <local> <remote>` all run over the same routed TCP session (adb docs).

- Security posture: over the network the connection is **more exposed than USB** and MITM-susceptible on untrusted networks — classic `adb tcpip` (Android ≤10) gates only on the previously-authorized RSA key and carries **no transport encryption**; use only on trusted networks (adb docs; DevGex ADB-over-TCP/IP guide). The L3 VPN **is** that trusted transport here — the WireGuard tunnel encrypts the hop, so routing 5555 inside the tunnel is the correct security envelope (**INFERENCE** from the two cited facts).

Tasks → sub-tasks

Task	Sub-tasks	Autonomous?
A1. Route reachability to device 5555	(a) bridge-up gate via `tests/lib/svord_bridge.sh` (`bridge_require`); (b) confirm `10/8` route exists on `ppp0` (svord owns the route — helix_proxy only *verifies*, never re-routes host tables autonomously, §11.4.133); (c) TCP-reachability of `HELIX_VPN_ADB_HOST:5555`.	Bridge-up = operator-gated; **gate + SKIP path autonomous**.
A2. `adb connect`	(a) `adb connect ${HELIX_VPN_ADB_HOST}:${HELIX_VPN_ADB_PORT:-5555}`; (b) accept `connected to` / `already connected to`; (c) capture `adb_connect.txt`.	Bridge-up only.
A3. State assertion	(a) `adb devices` → grep **only our serial** (§11.4.174); (b) require state `device` (not `offline`/`unauthorized`/absent); (c) reachable-but-unusable ⇒ **FAIL** (a real not-working state), absent ⇒ **SKIP**.	Bridge-up only.
A4. Debug round-trip	(a) `adb -s <serial> shell getprop ro.product.model` → non-empty model = the PASS evidence; (b) OPTIONAL extend (§11.4.146 STEP 3): `logcat -d` capture, `push`+`pull` byte round-trip with sha256 MATCH; (c) write `getprop.evidence`.	Bridge-up only.

The exact autonomous test that proves it

tests/vpn_lan/adb_over_vpn.sh (committed, Phase 7). Behaviour, cross-referenced from the source:

- **Env vars** (bridge contract, PLAN.md §3): `HELIX_SVORD_DIR`, `HELIX_BRIDGE_CONNECT`, `HELIX_BRIDGE_DISCONNECT`, `HELIX_BRIDGE_HEALTH`, `HELIX_BRIDGE_SUBNET`, `HELIX_BRIDGE_HOST`; ADB target `HELIX_VPN_ADB_HOST` (device `10.x` address), `HELIX_VPN_ADB_PORT` (default `5555`); test override `SVORD_BRIDGE_LIB`.
- **Honest-SKIP-when-bridge-down** (§11.4.3 / §11.4.69): the script's first action after sourcing the bridge library is `bridge_require`; a non-zero return prints `SKIP:network_unreachable_external` and `exit 0` **before `adb` is ever invoked** — so with the bridge down (the default autonomous state) **no device, and no operator/`lava-*` serial, is ever touched**. A down bridge is never a FAIL and never a fake PASS.
- **PASS gate:** `ab_pass_with_evidence` requires a non-empty evidence file — a non-empty `ro.product.model` captured in `qa-results/vpn_lan/phase7/<UTC-ts>/debug/getprop.evidence` (§11.4.69). Absent device/tool ⇒ SKIP; reachable-but-unusable ⇒ FAIL.
- **Evidence layout:** `qa-results/vpn_lan/phase7/<UTC-ts>/{connect,debug,flash}/`.

Honest boundary (§11.4.6): Phase A's *logic* (gate, SKIP path, evidence-gated PASS emitter) is autonomously provable now with the bridge down; the *live* connect/debug round-trip proves out only when the operator supplies the bridge + a configured `HELIX_VPN_ADB_HOST` (§8).

Phase B — ADB CONNECT model (server placement · TLS pairing · multi-device · safety) `[design + operator-gated live]`

Goal: define *where the adb server runs*, *how devices authenticate*, *how multiple devices are disambiguated*, and *the single-serial safety rule* — so the connect path is correct, secure, and cannot collaterally disrupt operator/`lava-*` devices.

B1. adb-server placement (the key VPN decision)

Two connection models exist; the deciding fact is **who owns the route to `10/8`** (adb docs server-on-5037 model; scrcpy #3848 the offline-over-VPN failure report):

- **Model 1 — adb server on the VPN-route-owning host (recommended).** Run the single `adb server` on the host that has the `ppp0` route into `10/8` (the svord-side / gateway host, or the containerized `vpn-lan-adb-server`, Phase D). It `adb connect`s to each remote device's 5555 over the direct route (no proxy hop). helix_proxy-side clients reach devices through that one server. This matches the committed test's "central adb server (one server, many remote devices)" note and the `containerization.md` ADB-server-helper design.
- **Model 2 — adb server on the proxy side + routed 5555.** The proxy-side `adb server` `adb connect`s straight to `10.x:5555` if the proxy host itself holds the `10/8` route. Works when the route is on the proxy host; the direct route (no forward hop) is what the committed test's T7.1 asserts.

Port-forward vs direct-route trade-off (INFERENCE from the cited server/route model): a direct L3 route (Model 1/2) is the clean fit — a single routed TCP session per device, full bidirectional adb multiplexing, no ALG rewriting. A `socat`/SSH port-forward of 5555 is only a fallback when no route exists; it pins one local port per device and complicates multi-device fan-out. **The route is preferred; the forward is the degraded fallback.** The observed "device shows `offline` over WireGuard" failure mode (scrcpy #3848) is the standard symptom of the adb server sitting on the wrong side of the tunnel (or a broken return route / MTU) — placing the server on the route-owning host (Model 1) is the structural fix (**INFERENCE**; the issue itself has no maintainer resolution, so this is a deduction from the adb server model, not a cited fix).

B2. Device authentication — RSA key + Android 11+ TLS pairing

- **All Android ≥4.2.2:** first connection prompts an on-device RSA-key authorization dialog; the device refuses adb until the key is accepted on an unlocked screen (adb docs). A network device stuck at `unauthorized` in `adb devices` means the key was not accepted — Phase A treats that as a not-usable state.
- **Android 11+ (API 30) wireless debugging:** a secure, USB-free path — pair with `adb pair IPADDR:PORT` and enter the on-device pairing code (or QR); every connection is authenticated by a 2048-bit RSA key-pair and encrypted with **TLS** (self-signed X.509), and services are discovered via **mDNS** (adb docs; Android 11 wireless debugging). Pairing uses a **separate dynamic port** from the 5555 connection port (adb docs).

- **VPN interaction with mDNS (INFERENCE):** mDNS discovery is **multicast** and does **not** cross the L3 VPN (PLAN.md §2 / reflector_design.md) — so on the proxy side you cannot rely on mDNS auto-discovery of the remote device; you `adb pair`/`adb connect` by the device's explicit `10.x:PORT` instead (the pairing/connect *unicast* traffic routes; only the *discovery* multicast doesn't). If mDNS discovery of remote devices is ever wanted, it needs the Phase-5 reflector — but for adb the explicit-IP connect is sufficient and is what the committed test does.

B3. Multi-device serial disambiguation

With many remote devices behind one server, **every** adb invocation MUST target a serial explicitly: `adb -s 10.x:5555 <cmd>` (adb docs). The committed test does exactly this — it references **only** `$ADB_TARGET` (`${HELIX_VPN_ADB_HOST}:${HELIX_VPN_ADB_PORT}`) and `awk`-matches **only** that serial in `adb devices`.

B4. The §11.4.174 single-serial safety rule (cross-ref the committed test's cleanup)

On a **shared** host the `adb devices` list may include operator devices and `lava-*` CI devices that are **off-limits** (§11.4.174 — verify ownership before acting). The committed test encodes the rule and this design ratifies it as mandatory for every adb-over-VPN operation:

- **NEVER `adb kill-server`** — it would tear down the shared server and drop every operator/`lava-*` device.
- **NEVER a blanket `adb disconnect`** (no-argument form) — it disconnects *all* network devices, not just ours.
- **Act on our serial only** — connect, state-check, and `getprop` all reference `$ADB_TARGET` exclusively; other serials in the list are never read and never acted on.
- **Cleanup disconnects only our serial** — the committed test's `trap cleanup EXIT INT TERM` runs `adb disconnect "$ADB_TARGET"` **gated on `ADB_CONNECTED=1`** (only if we actually made the connection), so the VPN device is never left connected and no foreign device is dropped (§11.4.14 + §11.4.174).

Tasks → sub-tasks

Task	Sub-tasks	Autonomous?
B1. Server placement	(a) decide Model 1 (server on route-owner) as default; (b) document the direct-route-preferred / port-forward-	Design autonomous.

Task	Sub-tasks	Autonomous?
	fallback trade-off; (c) cross-ref the containerized `vpn-lan-adb-server` (Phase D).	
B2. Authentication	(a) RSA-key authorization flow; (b) Android 11+ `adb pair` TLS/mDNS path + the multicast-does-not-cross-VPN caveat; (c) unicast connect-by-explicit-IP for VPN.	Design autonomous; live pairing operator-gated.
B3. Disambiguation	(a) mandatory `adb -s <serial>`; (b) single-target `awk` match.	Autonomous (mirrors committed test).
B4. Single-serial safety	(a) no `kill-server`; (b) no blanket `disconnect`; (c) our-serial-only cleanup trap gated on `ADB_CONNECTED`.	Autonomous (mirrors committed test).

Honest boundary (§11.4.6): server-placement + safety rules are settled design and mirrored in the committed test now; the live TLS-pairing round-trip (Android 11+) is operator-gated (needs a real device + on-device pairing-code entry, an `operator_attended` action).

Phase C — FLASH over VPN via usbip
`[operator-gated by physics + safety]`

Goal: define the *only* correct remote-flash path — carry the device's **USB** transport over the network with **usbip** — and wrap it in the §11.4.133 target-hardware-safety envelope. This is the committed test's T7.4 boundary: the test **never** flashes and **never** runs `fastboot`/`usbip`; it records the boundary and SKIPs `operator_attended`.

C1. Why fastboot is USB-bound on consumer devices

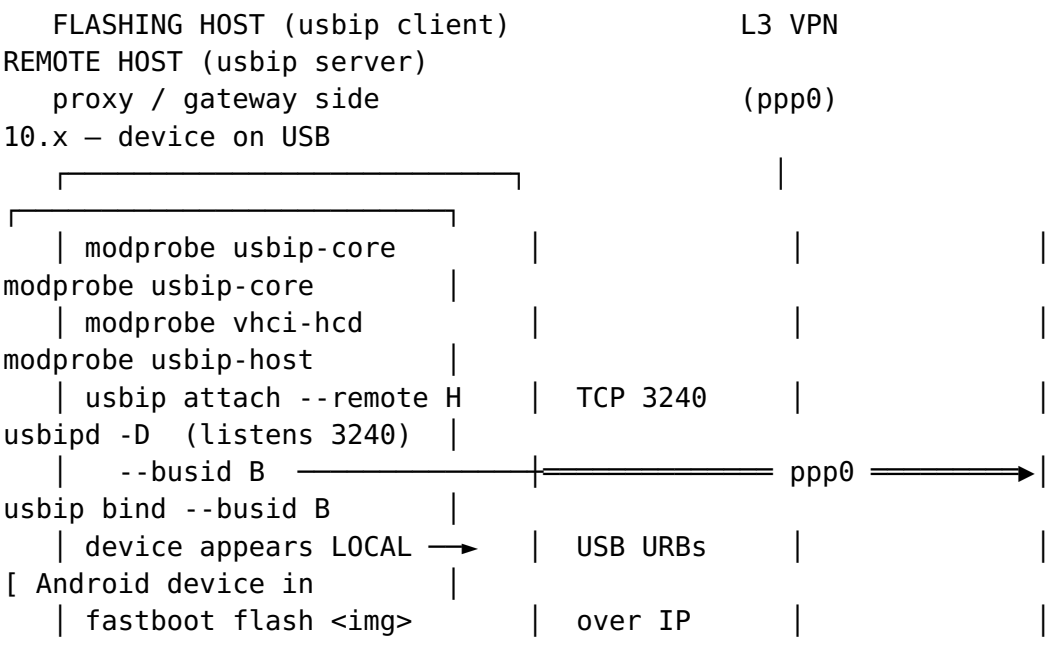
`fastboot` is a bootloader-stage protocol that is **host-driven and synchronous** with a `OKAY`/`FAIL`/`DATA`/`INFO`/`TEXT` response framing, defined "over USB or ethernet" (AOSP fastboot README). In the bootloader stage a consumer device exposes fastboot as a **USB gadget**; the ethernet transport requires bootloader support that

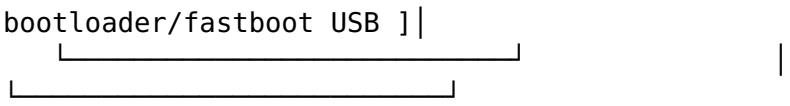
consumer devices generally do not ship, so a network `fastboot` command has no IP endpoint to reach on the device (**INFERENCE** — the over-ethernet capability is the cited FACT; its non-exposure on consumer bootloaders is the deduction). `fastbootd` (Android 10+ userspace fastboot) still speaks the same protocol over the device's USB gadget from userspace (AOSP Move fastboot to userspace). Therefore the routable path is to carry the **USB** transport itself over IP.

C2. usbip server/client topology

usbip is a Linux USB-over-IP system with a server/client split; the daemon listens on **TCP 3240** (USB/IP protocol — Linux Kernel docs; usbip README, kernel.org; RidgeRun USB/IP wiki). usbip is explicitly usable for **"flashing Android devices, using ADB and Fastboot"** (RidgeRun USB/IP wiki).

- **Server = the remote host with the device physically attached** (on the `10/8` subnet):
  - load modules: `modprobe usbip-core` / `modprobe usbip-host` (a.k.a. `insmod usbip-core.ko` / `usbip-host.ko`);
  - start daemon: `usbipd -D` (listens on **TCP 3240**);
  - enumerate + export the device: `usbip list -l` then `usbip bind --busid <busid>` (usbip README).
- **Client = the flashing host** (proxy/gateway side, reached over the routed VPN):
  - load modules: `modprobe usbip-core` / `modprobe vhci-hcd`;
  - discover + attach: `usbip list --remote <server>` then `usbip attach --remote <server> --busid <busid>`;
  - the device now appears **as a local USB device** on the client, so local `fastboot`/`adb` see it;
  - detach when done: `usbip detach --port <port>` (usbip README).





C3. usbip security + latency caveats

- **No built-in encryption/authentication (INFERENCE, high-confidence).** The kernel usbip README documents no auth/crypto in the protocol and the operational guidance it gives is firewall/SELinux posture, not confidentiality (usbip README; USB/IP protocol — Linux Kernel docs). The safe deployment therefore carries usbip **inside** the already-encrypted WireGuard L3 tunnel (never exposing 3240 to an untrusted network) and scopes 3240 to the VPN subnet via the same SSRF-allowlist discipline as every other port (`PLAN.md` §4). Marked INFERENCE because the "tunnel it over VPN/SSH" recommendation is standard practice deduced from the documented absence of usbip crypto, not a single cited sentence.
- **Latency / reliability caveat (INFERENCE from the protocol being synchronous URB-over-TCP).** usbip serializes USB request blocks over TCP; a lossy/high-latency link makes bulk USB transfers (a multi-hundred-MB flash) slow and increases the window in which a stall could interrupt a write. A flash over usbip should run over a stable, low-latency VPN segment, and the device should be on mains power — an interrupted flash is exactly the brick risk of C4. (Deduction from the synchronous host-driven protocol shape in the AOSP fastboot README + the URB-over-TCP usbip model.)

C4. §11.4.133 operator-gated safety envelope (no autonomous brick-risk flash)

Flashing can **brick** a device: an improperly-signed or mismatched boot image renders the device "unbootable and unrecoverable without unlocking the bootloader again," and relocking over mismatched software can trip anti-rollback into a hard brick (AOSP Lock and unlock the bootloader). Therefore, under §11.4.133 (target-hardware-safety) + §11.4.122 (no silent change to a connected host) + §11.4.66 (interactive-clarification), remote flashing is **operator-authorized only**, and any flow that ever executes it MUST:

1. **Ask first (§11.4.66/§11.4.122)** — an interactive options question before any flash; no autonomous flash decision.
2. **Backup first (§9.2)** — capture recoverable state (current slot / boot / partition backup where the device allows) before writing.

3. **Verify the image** — flash only a vendor-signed / verified image whose checksum is confirmed; never an unverified artifact (AOSP locking/unlocking).
4. **Stable transport + power** — usbip over a low-latency VPN segment, device on mains (C3).
5. **Never autonomously relock** — relock only after restoring vendor-signed stock images, operator-driven.

Tasks → sub-tasks

Task	Sub-tasks	Autonomous?
C1. USB-bound boundary	(a) document fastboot-protocol-over-USB/ethernet FACT + consumer-bootloader-USB-only INFERENCE; (b) record it in the test's `flash/boundary.evidence`.	Autonomous (design + boundary doc).
C2. usbip topology	(a) server: `usbip-core`/`usbip-host` + `usbipd -D` + `usbip bind`; (b) client: `usbip-core`/`vhci-hcd` + `usbip attach`; (c) port 3240; (d) device-appears-local then local `fastboot`.	**Operator-gated** (real device + remote-host change §11.4.122).
C3. Security + latency	(a) tunnel 3240 inside WireGuard, never exposed; (b) SSRF-allowlist scope 3240 to the VPN subnet; (c) stable/low-latency segment + mains power.	Design autonomous; deploy operator-gated.
C4. Safety envelope	(a) ask-first §11.4.66/§11.4.122; (b) backup §9.2; (c) verified image; (d) no autonomous relock.	**Operator-gated** (§11.4.133).

Honest boundary (§11.4.6): the committed test `adb_over_vpn.sh` **records this boundary and SKIPs `operator_attended`** — it never runs `fastboot`/`usbip` and never flashes. Everything in Phase C above the SKIP is *design*; the live flash is operator-authorized only and is never performed by any autonomous test.

Phase D — Containerization of the adb-server helper `[on-demand, operator-gated deploy]`

Goal: the central adb server (Phase B Model 1) runs as an **on-demand container** booted via the containers submodule — never by hand — cross-referencing the already-designed service.

The service is declared as **`vpn-lan-adb-server`** in `vpn_lan_containers.yaml` and designed in `containerization.md`: a central `adb server` fronting many remote devices, reachable over routed TCP 5555, so helix_proxy-side `adb connect 10.x:5555` needs no proxy hop. Its readiness is a **TCP health probe on 5555** (`nc -z 127.0.0.1 5555`), its image + bind address are **injected** (`${VPN_LAN_ADB_SERVER_IMAGE}`, `${VPN_LAN_ADB_BIND:-127.0.0.1}`) so no secret/literal lands in the submodule (§11.4.28 / §11.4.10).

Port-model clarification (INFERENCE, design refinement — does not contradict the committed artifacts): the adb *server↔client control* channel is TCP **5037** and stays container/host-local; **5555** is the *device-facing adbd* port the co-located adb server dials over the route. The `vpn-lan-adb-server` service's exposed 5555 is the routed device-facing port (as the yaml/containerization docs describe); the 5037 control port is not exposed off-host. This is the precise port split behind the "reachable over routed TCP 5555" phrasing in the existing docs.

Boot rules (from `containerization.md`, restated): started via `submodules/containers` `pkg/boot` / `pkg/compose` / `pkg/health` (rootless Podman, §11.4.161) — **no ad-hoc `podman`**; booting on a remote host **changes that host** so it is **operator-gated** (§11.4.122 / §11.4.66) — the local proof (`../../../tests/vpn_lan/container_boot.sh`) is a config-parse / plan-level readiness check that **starts nothing**.

Tasks → sub-tasks

Task	Sub-tasks	Autonomous?
D1. Service declaration	(a) `vpn-lan-adb-server` in `vpn_lan_containers.yaml` (image/ports/health, injected config only); (b) TCP health probe on 5555.	Autonomous (declaration + parse proof).
D2. On-demand boot	(a) `pkg/boot`/`pkg/compose`/`pkg/health`; (b) rootless Podman; (c) no ad-hoc podman.	**Operator-gated** (remote-host change §11.4.122).
	(a) `container_boot.sh` config-parse/plan check; (b) starts	Autonomous.

Task	Sub-tasks	Autonomous?
D3. Local plan proof	nothing; (c) honest-SKIP when bridge/runtime absent.	

3. Tasks / sub-tasks master table (per phase)

Phase	Task	Sub-tasks (condensed)	Autonomous vs Operator-gated	Proving te evidence
A access+debug	A1 route reachability	bridge gate · verify `10/8` route · TCP 5555 reach	gate/SKIP **autonomous**; live **op-gated**	`adb_over_vpn` (bridge-dow
A	A2 `adb connect`	connect · accept connected · capture log	op-gated (bridge up)	`connect/adb_`
A	A3 state assertion	`adb devices` our-serial only · require `device`	op-gated	`connect/adb_`
A	A4 debug round-trip	`getprop` model · optional logcat/ push-pull sha256	op-gated	`debug/getpro`
B connect model	B1 server placement	Model 1 route-owner default · route-vs-forward	design **autonomous**	this doc · `containeriza`
B	B2 authentication	RSA-key · Android 11+ `adb pair` TLS/ mDNS · unicast-connect-by-IP	design autonomous; live pairing **op-gated**	adb docs · t
B	B3 disambiguation	mandatory `adb -s <serial>` · single-target match	**autonomous** (mirrors test)	`adb_over_vpn`
B	B4 single-serial safety	no `kill-server` · no blanket `disconnect` · our-serial cleanup trap	**autonomous** (mirrors test)	`adb_over_vpn` cleanup (§1
C flash	C1 USB-bound boundary	fastboot-USB/ ethernet FACT · consumer-USB-	**autonomous** (boundary doc)	`flash/bounda`

Phase	Task	Sub-tasks (condensed)	Autonomous vs Operator-gated	Proving te evidence
		only INFERENCE · boundary evidence		
C	C2 usbip topology	server bind · client attach · port 3240 · local-device fastboot	**operator-gated** (§11.4.122/ §11.4.133)	design only SKIPs
C	C3 security+latency	tunnel 3240 in WireGuard · SSRF-scope · stable segment+power	design autonomous; deploy op-gated	this doc
C	C4 safety envelope	ask-first · backup · verified image · no autonomous relock	**operator-gated** (§11.4.133)	this doc · §1 question
D container	D1 service declaration	`vpn-lan-adb-server` yaml · TCP 5555 health	**autonomous** (parse proof)	`vpn_lan_con` · `container_`
D	D2 on-demand boot	`pkg/boot/compose/health` · rootless · no ad-hoc podman	**operator-gated** (§11.4.122)	`containeriza`
D	D3 local plan proof	config-parse/ plan · starts nothing · honest-SKIP	**autonomous**	`container_bo`

---

4. Honest boundary — proven-autonomously-now vs operator-gated (§11.4.6)

Proven autonomously now (no operator input, no secrets, touches no device):

- The bridge-down **honest-SKIP** path of `adb_over_vpn.sh` — `SKIP:network_unreachable_external`, `exit 0`, `adb` never invoked, no operator/`lava-*` serial touched.

- The **single-serial safety design** (Phase B4) — no `kill-server`, no blanket `disconnect`, our-serial-only cleanup gated on `ADB_CONNECTED` — mirrored in the committed test.
- The **flash-boundary record** (Phase C1) — `flash/boundary.evidence` documenting the USB-bound reality; the test SKIPs `operator_attended` and never runs `fastboot`/`usbip`.
- The **container plan-level proof** (Phase D3) — `container_boot.sh` parses the boot plan and starts nothing.

Operator-gated (parked, §11.4.21 / surfaced via §11.4.66 when reached):

- The **live** `adb connect` + debug round-trip (Phase A2–A4) — needs the svord bridge up + a configured `HELIX_VPN_ADB_HOST` on a real device.
- **Android 11+ TLS pairing** live round-trip (Phase B2) — on-device pairing-code entry is `operator_attended`.
- **Any usbip flash** (Phase C2/C4) — real device + remote-host change (§11.4.122) + brick-risk (§11.4.133): backup + verified image + interactive approval, never autonomous.
- **Container deploy on a remote host** (Phase D2) — changes that host; operator-approved boot only.

This document guarantees a **correct, cited, security-reconciled, evidence-anchored** design for full ADB-over-VPN (access, debug, connect, flash). It does **not** claim any live device result — every live path is honestly SKIPped by the autonomous test until the operator supplies the bridge, and flashing is operator-authorized only. No phase is "done" until its runtime signature verifies with captured evidence (§11.4.108) and it crosses independent review (§11.4.142) + the §11.4.169 test-type matrix.

Sources verified 2026-07-01

- Android Debug Bridge (adb) — official Android developer docs (adb server port 5037, device `adbd` TCP 5555, `adb tcpip 5555`, `adb connect IP:5555`, `adb devices` states, `adb -s <serial>`, `adb kill-server`, `adb disconnect`, Android 11+ `adb pair` TLS + mDNS, RSA-key authorization, network security posture): https://developer.android.com/tools/adb
- Android 11's Wireless debugging (adb pair, 2048-bit RSA key-pair, self-signed X.509 TLS, mDNS): https://medium.com/@urvesh/android-11s-wireless-debugging-5d0f6448ee3
- ADB over TCP/IP guide (classic `adb tcpip` / `adb connect`, trusted-network security): https://devgex.com/en/article/00000826
- scrcpy connection doc (`--tcpip`, `--serial=IP:5555`, device reboot disables `adb tcpip`, Android 11 wireless-debugging bypass): https://github.com/Genymobile/scrcpy/blob/master/doc/connection.md

- scrcpy + ADB-over-WiFi (reboot disables TCP/IP mode; USB needed once for classic path): https://proandroiddev.com/supercharge-android-dev-with-scrcpy-and-adb-wifi-f286091c72fc
- scrcpy issue #3848 (adb device shows `offline` over a WireGuard VPN — the interplay failure mode; no maintainer resolution): https://github.com/Genymobile/scrcpy/issues/3848
- USB/IP protocol — The Linux Kernel documentation (usbip protocol, TCP 3240): https://docs.kernel.org/usb/usbip_protocol.html
- usbip README — kernel.org (`usbip-core`/`usbip-host`/`vhci-hcd` modules, `usbipd -D`, `usbip list -l`/`--remote`, `usbip bind --busid`, `usbip attach --remote --busid`, `usbip detach`, TCP 3240): https://www.kernel.org/doc/readme/tools-usb-usbip-README
- RidgeRun USB/IP wiki (usbip for "flashing Android devices, using ADB and Fastboot"; port 3240): https://developer.ridgerun.com/wiki/index.php/How_to_setup_and_use_USB/IP
- AOSP fastboot README (fastboot protocol "over USB or ethernet", host-driven synchronous, OKAY/FAIL/DATA/INFO framing): https://github.com/aosp-mirror/platform_system_core/blob/master/fastboot/README.md
- AOSP Move fastboot to userspace / fastbootd (userspace fastboot over the device USB gadget): https://source.android.com/docs/core/architecture/bootloader/fastbootd
- AOSP Lock and unlock the bootloader (improperly-signed boot image ⇒ unbootable/unrecoverable; relock brick risk / anti-rollback): https://source.android.com/docs/core/architecture/bootloader/locking_unlocking

Access date for all sources: 2026-07-01. INFERENCE items are explicitly marked in-text (§11.4.6): the consumer-bootloader-exposes-fastboot-over-USB-only deduction (Phase C1), the tunnel-usbip-inside-the-VPN security posture (Phase C3), the adb-server-on-the-route-owner fix for the offline-over-VPN symptom (Phase B1), the mDNS-does-not-cross-L3 adb-discovery caveat (Phase B2), and the 5037-vs-5555 port-split clarification (Phase D) — each a deduction from the cited facts, not itself a cited fact. Deep-research multi-angle (§11.4.150): protocol-layer (adb/fastboot/usbip specs), routing-layer (L3 VPN carries unicast TCP, not USB, not multicast), security-layer (RSA/TLS pairing, usbip-has-no-crypto, brick-risk), safety-layer (operator-gated flash §11.4.133).